

Guide auteur, créer et modifier le contenu

Ce guide s'adresse à l'enseignant ou à l'auteur du TP. Il explique comment ajouter, modifier, réordonner et retirer des niveaux (les exercices), depuis l'appli, sans toucher aux fichiers à la main. Le format du contenu est décrit en fin de document pour qui préfère éditer directement.

1. Le mode auteur

L'édition du contenu est protégée par un mot de passe, pour qu'un étudiant ne modifie pas le parcours par curiosité.

- **Mot de passe par défaut** : `auteur` . À changer dès la première prise en main : menu **Paramètres -> Changer le mot de passe auteur**.
- Le mot de passe n'est jamais stocké en clair (empreinte SHA-256, dans `auteur.json` , local au poste et non partagé).

2. Vocabulaire

- **Parcours** : une suite d'exercices, un dossier sous `contenu\` (par exemple `be_c`).
- **Niveau** (ou exercice) : une étape du parcours, un sous-dossier avec son énoncé, son code de départ, son corrigé.
- **Porte** : le test qui valide un niveau. Pour un exercice « programme », la porte compile le code et vérifie que la sortie contient bien la **sortie attendue**.

3. Gérer les niveaux depuis l'appli

Menu **Paramètres -> Gérer les niveaux** (mot de passe demandé). La fenêtre liste les niveaux actifs dans l'ordre du parcours.

- **Ajouter** : formulaire (identifiant, titre, mode, fichier édité, cran débloqué, nœud de cours, sortie attendue). À la validation, le dossier et ses fichiers de départ sont créés, et l'appli propose de remplir le niveau tout de suite.
- **Modifier** : ouvre l'éditeur de contenu du niveau sélectionné. Un onglet par fichier (`enonce.md` , `starter.c` , `corrige.c`), plus le titre et la sortie attendue. Le bouton Enregistrer écrit dans les fichiers.
- **Retirer** : **détache** le niveau du parcours. Le dossier n'est pas effacé, il sort seulement de la liste. Rien n'est perdu.
- **Monter / Descendre** : change la place du niveau dans le parcours.
- **Détachés** : liste les niveaux détachés (encore sur le disque, hors du parcours) et permet d'en réattacher un.

Bon réflexe après un ajout

Les fichiers créés sont des gabarits vides. Remplis, via **Modifier** :

1. `enonce.md` : l'énoncé vu par l'étudiant.
2. `starter.c` : le code de départ (incomplet, il ne doit **pas** passer la porte).
3. `corrige.c` : le corrigé de référence (il **doit** passer la porte).
4. La **sortie attendue** : une ligne par fragment qui doit apparaître dans la sortie du programme.

La règle d'or : le corrigé franchit la porte, le starter non. C'est ce qui garantit que la porte teste vraiment quelque chose.

4. Changer de parcours, ouvrir le contenu

- **Paramètres -> Changer de parcours** : liste les parcours disponibles et mémorise le choix. Le changement prend effet au prochain lancement (ferme puis relance l'appli).
- **Paramètres -> Ouvrir le dossier du contenu** : ouvre l'Explorateur sur le dossier du parcours courant, pour éditer les fichiers à la main si besoin.

5. Format du contenu (pour l'édition manuelle)

Un parcours est un dossier `contenu\<nom>\` contenant :

- `parcours.json` : l'ordre des niveaux et le mode.
- un sous-dossier par niveau.

`parcours.json` :

```
{
  "ordre": ["ex01_types", "ex02_operateurs"],
  "mode": "isole"
}
```

Un niveau `contenu\<nom>\ex01_types\` contient :

Fichier	Rôle
<code>meta.json</code>	Métadonnées du niveau (voir ci-dessous)
<code>enonce.md</code>	L'énoncé affiché (Markdown)
<code>starter.c</code>	Le code de départ
<code>corrige.c</code>	Le corrigé de référence
<code>approfondissement.md</code>	Optionnel, contenu supplémentaire du niveau

`meta.json` :

```
{
  "id": "ex01_types",
  "titre": "Exercice 1, les types de variables",
  "type": "programme",
  "mode": "programme",
  "cran_debloque": 1,
  "noeud_cours": "Exercice 1 du BE C, les types de base, slides 8 a 10",
  "fichier_edite": "programme.c",
  "sortie_attendue": ["short : 12", "int : 260", "char : A"]
}
```

Champ	Sens
<code>id</code>	Identifiant, doit valoir le nom du dossier
<code>titre</code>	Titre affiché
<code>type</code> / <code>mode</code>	<code>programme</code> pour un exercice programme complet
<code>cran_debloque</code>	Cran d'aide tuteur débloqué à la validation (0 a 3)
<code>noeud_cours</code>	Rappel du point de cours, sert au tuteur
<code>fichier_edite</code>	Nom du fichier que l'étudiant édite
<code>sortie_attendue</code>	Fragments qui doivent figurer dans la sortie

L'appli et l'outil auteur écrivent le même format : on peut mélanger édition à la main et édition par la fenêtre sans rien casser.

6. Bon à savoir

- Une porte « programme » ne s'ouvre que si le programme compile **et** que sa sortie contient tous les fragments de `sortie_attendue`.

- Un identifiant de niveau est en minuscules, chiffres et « _ » seulement, et commence par une lettre (exemple : `ex14_boucles`).
- Le compilateur `gcc` est nécessaire pour tester une porte. Vérifie sa présence dans **Paramètres -> Emplacements et diagnostic**.